

Chapter 4

Deep Learning

Deep learning emerged as a serious contender in the field from a 10-year explosive computational development. Deep learning is therefore a special form of machine learning, the algorithms of which are based on the structure and function of the human brain.

Machine Learning VS Deep Learning

Deep learning is arguably the most efficient form of machine learning. This is so important because they know the best way to solve the problem and how to fix everything. The following is a description of deep learning and machine Learning:

- **Data Dependency:** The first difference point is based on the DL and ML output as the data scale increases. Deep learning algorithms are very good when the data is large.
- **Machine Dependency:** High-end machines allow deep learning algorithms to function perfectly. Machine learning algorithms, on the other hand, can also operate on low-end computers.
- **Machine dependency:** Deep learning algorithms can extract features of high level and seek to learn from them. At the other hand, the majority of features derived from machine learning are identified by an expert.
- **Time of Execution:** Runtime depends on the various algorithm parameters. Deep learning has greater parameters than algorithms for machine learning. Therefore, the runtime of DL algorithms is more than ML algorithms, especially the training time. However, the DL algorithms 'processing time is less than ML.
- **Problem solving Approach:** Deep learning addresses the end-to-end problem when the machine Learning is used to solve the problem historically, i.e., by splitting it into bits.

Basic Terms in Deep Learning

- **Neurons:** Just as a neuron forms our brain's fundamental feature, so a neuron forms the fundamental structure of a neural network. Think about what we do when we obtain new information. We process it and then generate an output when we get the information. Similarly, if a neural network is involved, as shown in the Fig 4.1 the input of a neuron is obtained and processed and generated by an output sent for further processing to other neurons or the final output.
- **Weights:** The input is multiplied by weight as applied to the neuron. For instance, if a neuron has two inputs, the weight is allocated to each input. We randomly initialize weights and change these weights during the model training. After preparation, the neural network gives more weight to the input it finds more important than the less important ones. A weight of zero refers to the insignificance of the particular element.

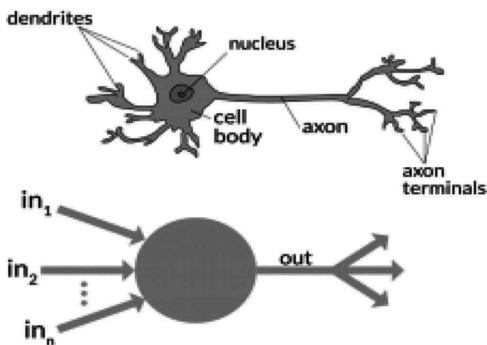


Fig. 4.1 Structure of Neuron

- **Bias:** Apart from the weights, the input is applied with a further linear component, called the bias. The result of weight propagation is added to the input. Basically, the distance of the input multiplied by weight is applied to alter. The outcome will look like $a*W_1 + \text{bias}$ after introducing this bias. This is the final linear dimension of the transformation in the data.
- **Activation Function:** A non-linear function is introduced when the linear component is added to the data. The activation function converts the input signals to output signals. Applies this function to the linear relation. The result after activation will look like $f(a*W_1 + b)$, where f is the activation function.

The inputs given under X_1 to X_n and according to the weight W_{k1} to W_{kn} are given as “ n ” in the diagram below. We’ve got a preference given as b_k . First, weights are multiplied by their respective input and then added together. Let’s call this like u .

$$u = \sum w * x + b$$

Accessible for u i.e., $f(u)$ is the activation function and we get the final neural output of

$$y_k = f(u)$$

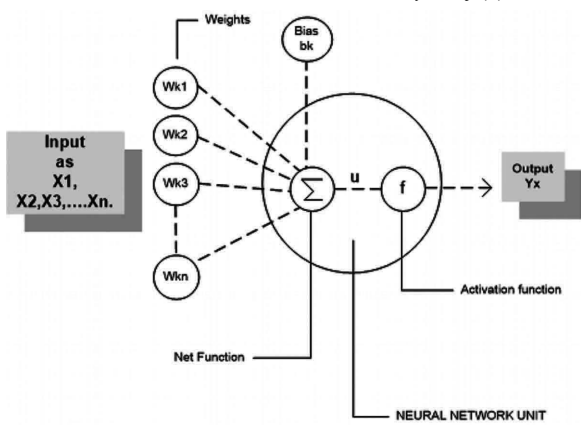


Fig. 4.2 Activation Function

The most commonly used activation functions are sigmoid, ReLU and SoftMax

- **Input/Output/Hidden layer:** As the name suggests, the input layer is the first layer of the network that receives the input. The output layer is the output layer or is the final network layer. The transmission layers of the network is the hidden layers. These secret layers are those that perform different tasks on the input data and transfer the output to the next layer. The layers of entry and output can be seen while the intermediate layers are covered and all this is represented in Fig 4.3.

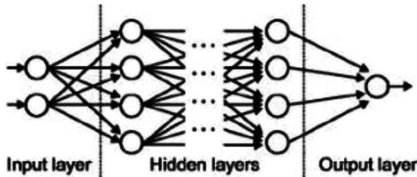


Fig. 4.3 Representation of layers

- **Multilayer Perception:** A single neuron cannot accomplish highly complex tasks. Therefore, we generate the necessary outputs with neuron stacks. We have an input, a hidden layer and an output layer on the simplest network. Growing layer has multiple neurons and in each layer are all neurons connected to the next layer of neurons. These networks can also be called fully connected.
- **Cost Function:** The network tries to estimate how close the performance is to the actual value when we create a network. We use the cost/loss function to calculate this precision in the network. If the network makes errors the cost or loss function attempts to penalize.

Our goal is to enhance our predictability and reduce error while running the network and thus decrease its costs. A minimal cost or loss function is the most efficient performance.

If the cost function is defined as the medium squared error, it can be written as:

$$C = 1/m \sum (y - a)^2$$

where m is the number of training inputs, a is the predicted value and y is the actual value of that particular example. The cycle of learning is about reducing costs.

- **Gradient Descent:** The gradient descent is an algorithm of optimization for cost reduction. To think about it intuitively, you would walk down a hill and go down rather than just run down immediately. So what we're doing in the Fig 4.4, we'll step down from a point x , i.e., Δh , to $x - h$ and we've got to do the same until the edge. To find the local minimum of one function, take steps proportional to the negative of the gradient of the function, consider the bottom as the minimum cost point.
- **Learning Rate:** In each iteration, the learning rate is defined as the level of cost minimization. The rate at which we go down to the minimum of cost function is simply the rate of learning. We should carefully select the rate of learning, as the optimum solution should not be skipped or incredibly low to be converged in the network forever which can be Represented as shown in Fig 4.5.

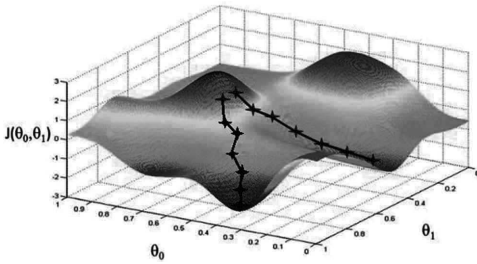


Fig. 4.3 Representation of layers

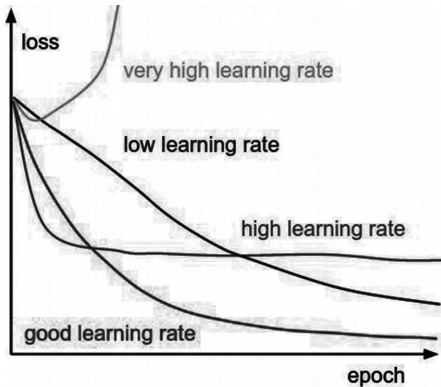


Fig. 4.5 Representation of Learning Rate

- **Back propagation:** We allocate a random weight and bias to our nodes when we construct a Neural Network. We can measure the network error once we have obtained the data for one iteration. This error is then returned to the network with the cost function gradient to correct the network weights. Such weights are then modified in order to reduce the errors in the following iterations. This weight change is called back propagation with the cost function gradient.

In the meantime, the network movement is reversed, the error and the gage is extracted from the hidden layers from the outer side and the weights are changed.

- **Batches:** During the course of a neural network we divide the input into multiple parts of equal size by chance, instead of sending the entire input in a single go. The batch data training makes the model more scalable when the entire data set is fed in one go into the network as compared with the model developed.
- **Epochs:** An epoch is characterized as a single iteration for all lots of propagation in the forward and backward. That means that one epoch is a reverse pass of the complete input data.

You can select the number of epochs to train your network. More epochs are more likely to show higher network precision, but it will take longer to converge on the Network. You will need to make sure the network is over-fit if the number of epochs is too small.

- **Dropout:** Dropout is a technique of regularization that avoids network over fitting. As the name indicates, a number of neurons in the hidden layer are lowered randomly during training. This means that training takes place for various neuron combinations in many architectures in the neural network. You may consider dropping out as an ensemble process, which then generates the final result using the output of multiple networks and this is demonstrated in Fig 4.6.

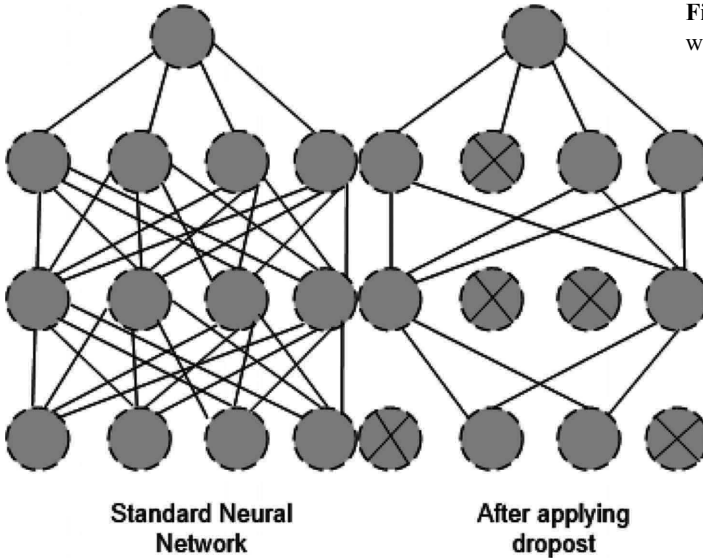


Fig. 4.6 Demonstrating working of dropout

- **Batch Normalization:** Batch normalization can be viewed as a process that we have placed in a river as a special control point. This is done to ensure the data distribution is the same as the next layer. The weights change after each gradient descent phase when we training the neural network. This modifies the way the data form is sent to the next layer. But, the next layer expected the distribution to be like what it had seen before. So, before sending it to the next stage, we specifically normalize the data.
- **Training:** Weights begin as random values, and as the neural network becomes familiar with what input data type is allowed, the network changes weights based on any categorization errors arising from the previous weights. The neural network is referred to as preparation. Once the network has learned, the result for the related input can be predicted by us.

Neural Networks

Neural networks are computer parallel devices which attempt to construct a brain computer model. The main objective is to develop a system which allows several computer tasks to be performed faster than traditional systems. These tasks include patterns recognize and identify, estimate, optimize and cluster data.

Artificial Neural Networks (ANN)

An effective computing system with a core theme taken out of the analogy with biological neural networks are an artificial neural network (ANN). Artificial Neural

Systems, Parallel Distributed Processing Systems and Connectionist Systems are also known as ANNs. In order to allow contact between them, ANN acquires a wide set of units linked in some pattern. Such units are simple processors which work in parallel, also known as nodes or neurons.

Every neuron is linked by a connecting link with another neuron. Each connection link is linked to a weight with input signal information. This is the most useful knowledge for neurons to solve a particular problem, as weight usually activates or inhibits the signal. The inner state of each neuron is called the activation signal. After combining input signals and activation rule, the output signals produced can be transmitted to other units.

Model of Artificial Neural Network:

The diagram below represents the general ANN model and its procedure.

Diagram

For the above general model of artificial neural network, the net input can be calculated as follows:

$$y_{in} = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 \dots x_m \cdot w_m$$

i.e., Net input $y_{in} = \sum_i m_i x_i w_i$

The output can be calculated by applying the activation function over the net input.

$$Y = F(y_{in})$$

Output = function net input calculated

Processing of ANN depends upon the following three building blocks –

Network Topology

The arrangement of a network with its linking nodes and lines is a network topology. ANN can be categorized as the following forms according to topology:

Feed forward Network: It is a non-recurring network with layer processors and all layer nodes are connected to previous layer nodes. The relation has different weights. Feedback loop does not occur so that the signal can only pass from input to output in one direction. The following two forms can be subdivided.

Single Layer Feed forward network: The definition has only one weighted layer of Feed forward ANN. This is, the input layer can be assumed to be fully connected to the output layer.

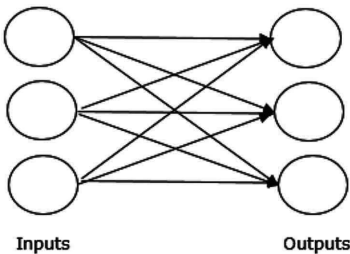


Fig. 4.7 Single Layer Feed forward Network

Multiple layer feedforward Network: The theory has more than one weighted layer of feedforward ANN. This network is called hidden layers because it has one or more layers between the input and output layer.

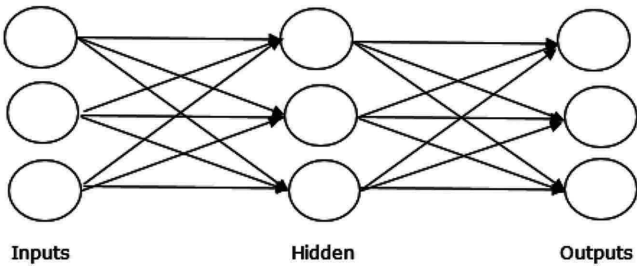


Fig. 4.8 Multiple Layer Feedforward Network

Feedback Network: As the name implies, a feedback network has feedback paths, indicating that the signal will flow through loops in any direction. This makes it a non-linear dynamic system that constantly changes before a balance is reached. The following forms can be separated:

Recurrent Networks: These are closed-loop feedback networks. The two forms of recurrent networks are as follows.

Fully Recurrent Network: This is the most basic neural network architecture because all nodes are connected to all of the other nodes. As can be seen in the below figure Fig 4.9.

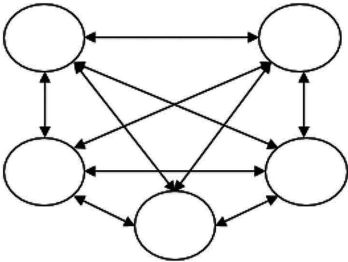
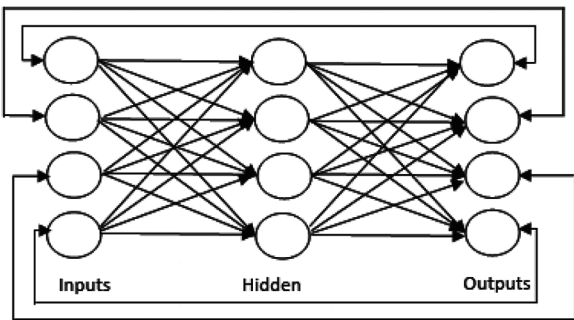


Fig. 4.9 Fully Recurrent Network

Jordan Network: It is a closed-circuit network where the output is transmitted to the input as feedback in the Fig. 4.10 below.



Adjustments of Weights

or Learning

The method of modification of the weights of connections between the neurons of a given network is learning in an artificial neural network. ANN learning can be categorized into three groups, which include supervised, unsupervised and Reinforcement learning. As we already studied about these learning in before chapter here we will discuss about how ANN uses these learning.

Supervised Learning: This form of education is, as the name implies, under the guidance of a teacher. This is a dependent learning process.

The input vector is introduced to the network during ANN's training under supervised learning, providing an output vector. Compared with the appropriate output vector this output Vector is. If the actual output and the desired output vector vary, an error signal is produced. The weights are modified according to this error signal until the real output matches the desired output as seen in the Fig. 4.11.

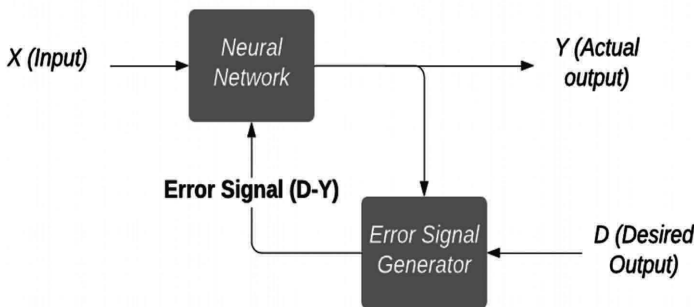


Fig. 4.11 Dependent Learning Process

Unsupervised Learning: This method of learning, as the name implies, is performed without an instructor's supervision. This is an individual learning process.

The input vectors of the same form are coupled with clusters in the training of ANN under uncontrolled instruction. The neural network will provide an output response indicating the class to which an input pattern belongs, if a new input pattern is added.

There is no environmental input as to what the optimal output will be, and whether it is right or wrong. Therefore, as seen in the Fig 4.12 the network itself must discover the patterns and features of input and the relationship between input and output for this form of learning.

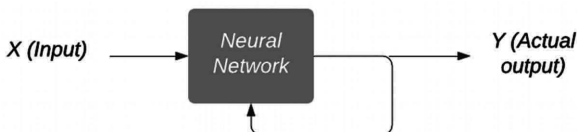


Fig. 4.12 Individual Learning Process

Reinforcement Learning: This kind of learning, as the name implies, is used for strengthening or improving the network through critical data. This learning process is similar to guided learning, but we could have far less knowledge.

The network provides some input from the community during training of the network. This makes the supervised learning very similar. However, the input received

here is not descriptive and evaluative, because no instructor exists as in supervised education. The network will change the weights to optimize important information in the future after receiving the feedback as it can be seen in the Fig 4.13.

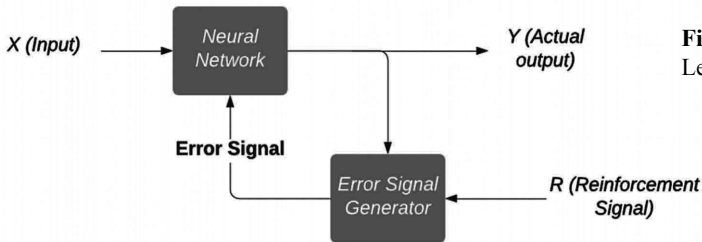


Fig. 4.13 Reinforcement Learning

Activation Functions

The additional force or effort over the input can be specified to obtain an accurate output. In ANN, activation functionality can also be added to achieve the same result through the input. Below are some important activation functions

Linear Activation Function: The identity function is also named as it does nothing to edit the data. It is possible to define:

$$F(x) = x$$

Sigmoid Activation Function: These are of two types namely:

Binary Sigmoidal Function: Input editing takes place between 0 and 1 this activation feature. By fact, it is optimistic. It is always limited, meaning that its performance is not less than 0 and not more than 1. It also increases purely in nature, meaning that the higher the input the output. You may describe it as

$$F(x) = \text{sigm}(x) = 1 \div 1 + \exp(-x)$$

Bipolar Sigmoidal Function: This activation function adjusts the input from -1 to 1. In fact, it can be positive or negative. It is always limited, meaning that its production cannot be below -1 and above 1. This also rises purely in nature such as sigmoid. It can be set as

$$F(x) = \text{sigm}(x) = 2 \div 1 + \exp(-x) - 1 = 1 - \exp(x) \div 1 + \exp(x)$$

Challenges in ANN

The first step in resolving the question of an image classification using ANN is to transform a 2D image into a 1D vector before training. There are two inconveniences:

- When the image size increases, the number of trainable parameters increases dramatically
- ANN loses the image's spatial characteristics. Spatial characteristics apply to the pixel arrangement in a image.

ANN cannot record sequential information for the processing of sequence data in input data.

Convolutional Neural Network

Convolution neural network is the same as regular neural network as it consists of neurons with learning weights and prejudicial characteristics. Normal neural networks ignore the input data structure and convert all data to a 1-D array before they are transferred into a network. This method corresponds to standard data, but the process can be tedious if the data includes images. This problem is easily solved by CNN. It takes into account the 2D structure of the images when processing them, enabling them to extract image-specific properties. The main aim of CNNs is therefore to transfer from raw image data in the input layer into the correct class in the output layer. The only difference in the treatment of incoming data and layer sort is between ordinary NNs and CNNs.

Architecture Overview of CNNs

The standard neural networks are architecturally assisted with an input and transform it through a hidden layer. With the support of neurons, each layer is related to the other layer. The main drawback of ordinary neural networks is that they do not reach full images correctly.

The CNN architecture is structured with neurons called width, height and depth in three dimensions. The current layer of each neuron is linked to a small piece of the preceding layer output. The overlay of a filter N to N on the input image is identical. M filters are used to ensure that all the information is obtained. These M filters are extracting elements such as sides, corners, etc.

Layers Used to Construct CNN

- **Input Layer:** It takes the raw image data as it is.
- **Convolutional Layer:** This layer is the central component of the CNNs where most computations are performed. This layer measures the convolutions in the input between the neurons and the specific patches.
- **Padding:** Filter also doesn't suit the input image perfectly. We have two alternatives:
 - Pad photos with zero padding (zero padding) to fit.
 - Delete the picture component that didn't match the filter. This is known as a valid padding that only holds valid part of the image
- **Rectified Linear Unit Layer:** It is used to activate the previous layer output. It increases the network's nonlinearity to allow for any form of function to be well generated. Because real-world data are non-negative linear values, our Network will want to know.
- Other non-linear functions are also possible, for example tanh or sigmoid, instead of ReLU. Many data scientists use ReLU because the ReLU is better than the two others.
- **Pooling Layer:** Pooling just helps to preserve the essential parts of the network as we progress. The pooling layer acts on each input slice independently and spatially resizes it. Spatial pooling, which reduces the dimensionality of each map but retains essential details, is often called subsampling or down sampling. Different forms of spatial pooling may be:

- Max Pooling
- Average Pooling
- Sum Pooling

Max pooling takes from the corrected feature map the largest element. The average pooling will take the biggest element as well. Description of all elements in the function map

- Fully Connected Layer/Output Layer: We flatter our matrix and feed it to a completely connected layer such as a neural network, as a layer called an FC layer. The output values in the last level are calculated by this layer. The resulting output is $1 \times 1 \times 1 \times L$ in which L is the dataset number training groups.

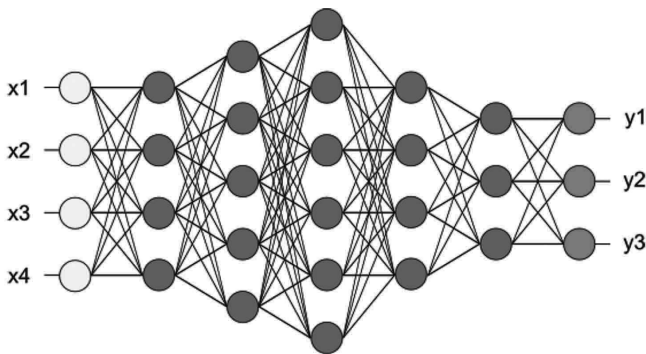


Fig. 4.14 After pooling layer, flattened as FC layer

The function map matrix is transformed as a vector in the above Fig 4.14 (x_1, x_2, x_3 , etc). We combined these features to create a layout with the fully connected layers. Eventually, we have an activation function for classifying outputs, such as SoftMax or sigmoid.

We can clearly understand the full architecture of CNN from the Fig 4.15.

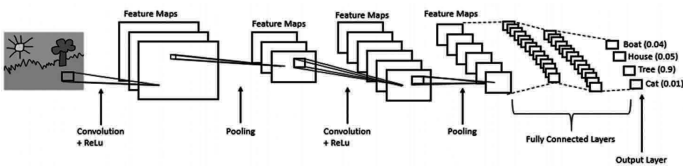


Fig. 4.15 Complete CNN architecture

Recurrent Neural Network

A recurring neural network (RNN) is an artificial neural network class, where a direct graph along a sequence relation forms a unit. This allows complex time actions to be exhibited for a period of time. As compared to neural feed forward networks, internal memory of RNNs may be used to process input sequences. This makes

them applicable to tasks like the unsegmented, related recognition of handwriting or language.

If every model requires a context to provide the output based on the data, the recurrent Neural Network comes into action. The context is often the most important thing for the model to predict the best performance.

Let's get an analogy to understand. Suppose you watch a movie, you continue to watch it because at all times, you have a history as you watched the movie before so, then you can only relate it all correctly. This means you know what you've been watching.

Similarly, RNN remembers everything. Both inputs and output are distinct from each other in other neural networks. In RNN, however, all inputs are connected. for instance, the relationship between all the terms helps to predict the next word in a given sentence which can lead to better output. During the training itself, the RNN recalls all these ties.

To do this, the RNN creates networks with loops that allow information to persist as in Fig 4.16.

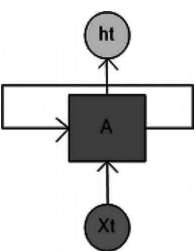


Fig. 4.16 RNN with a LOOP

The neural network will sequence input from this loop structure. You'll understand it better if you see the unrolled structure of network in the Fig 4.17.

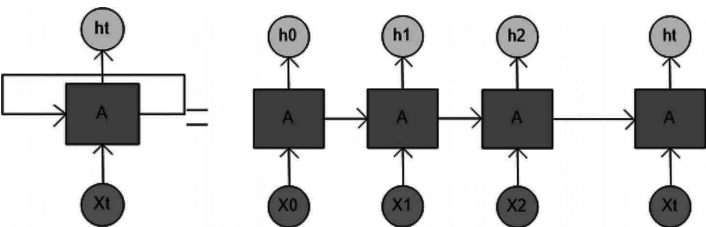


Fig. 4.17 Unrolled Structure of RNN

As in the unrolling structure you can see. First of all, the $x(0)$ is taken from the input sequence and then $h(0)$ is generated, which is the input together with $x(1)$. The input for the next step is $h(0)$ and $x(1)$. Similarly, $h(1)$ for the next step and so on is the entry with $x(2)$. It preserves its meaning during training.

“Whenever there is a data series and the temporal dynamics connecting the data are more important than the spatial content of each frame.”

Types of RNN's

The key explanation for the more thrilling recurring networks is that it allows one to operate on sequences of vectors: input sequences, output sequences or in most cases both. This could be more specific with a few examples:

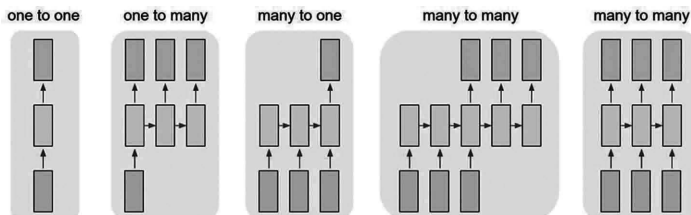


Fig. 4.18 Types Of RNN

Each rectangle in above image represent Vectors and Arrows represent functions. Input vectors are Red in color, output vectors are blue and green holds RNN's state.

One to One: It often named Plain neural networks. It addresses Fixed input size to Fixed output size where it is independent of previous data/output.

Example: Classification of images.

One to many: This deals with information of a fixed size as input and provides data sequence as output.

Example: Image tagging takes the image as input and shows a word sentence.

Many to One: It takes information sequence as an input and passes a certain output scale.

Example: sentimental analysis where the sentences is positive or negative is graded.

Many to Many: It takes an information sequence as input and processes it repeatedly produces a data set.

Example: Machine translation where an RNN reads an English phrase and then publishes a French phrase.

Bidirectional Many to many: Input and output synced sequence. Note that no pre-specified length sequence restrictions are present in any case, since repeating (green) transformations are fixed and can be applied as much as we want.

Example: video grading where each video frame is named.

Challenge of RNN

Deep RNNs (RNNs with several time steps) are often affected by the disappearance and exploding problems in gradients which are typical in all the neural network types.

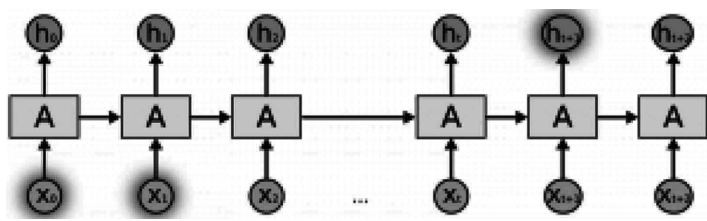


Fig. 4.19 Recurrent Neural Network

As you can see in the Fig 4.19, the calculated gradient will disappear when the initial time stage is reached.

Difference Between ANN, CNN & RNN

We will depicts all the differences between Artificial Neural Network, Convolution Neural Network and Recurrent Neural network using a table for better understanding.

	ANN	CNN	RNN
Data	Tabular data	Image data	Sequence data
Recurrent Connections	No	No	Yes
Parameter Sharing	No	Yes	Yes
Spatial Relationship	No	Yes	No
Vanishing and Exploding Gradient	Yes	Yes	Yes

Summary

In this chapter we have studied about the basics of deep learning which includes the terminology or the basics terms we address while using the deep learning or any other algorithms like bias, learning rate etc., and we have also studied about the artificial neural network, Convolutional Neural Network and Recurrent Neural Network with their basics structure which can be used in different scenarios according to the requirement of the problems. In the Implementation part of this book we will be using CNN for classification and try to recognize the plant or their species.